

JAVA WEB

Unit 7: Spring Security



Mục lục

1. Spring Security
 - Giới thiệu
 - Bổ sung xác thực, phân quyền cho ứng dụng: SpringBootJpaJsp
2. Giới thiệu OAuth2
3. Giới thiệu JSON Web Token (JWT)



Spring Security

Spring Security là một loạt các bộ lọc servlet giúp chúng ta thực hiện authentication và authorization ứng dụng web.

Spring Security tích hợp tốt với Spring Web MVC, Spring Boot, giao thức OAuth2, JSON Web Token (JWT).



Bổ sung xác thực, phân quyền cho Project: SpringBootJpaJsp

Thêm các Dependency vào pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<!-- https://mvnrepository.com/artifact/org.springframework.security/spring-security-taglibs -->
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-taglibs</artifactId>
  <version>5.4.5</version>
</dependency>
```

Tạo bảng và chèn 02 bản ghi vào bảng users

```
CREATE TABLE `users` (
  `user_id` int(11) NOT NULL AUTO_INCREMENT,
  `username` varchar(45) NOT NULL,
  `password` varchar(64) NOT NULL,
  `role` varchar(45) NOT NULL,
  `enabled` tinyint(4) DEFAULT NULL,
  PRIMARY KEY (`user_id`)
);

INSERT INTO `users` (`username`,`password`,`role`,`enabled`)
VALUES ('test', '$2a$10$G3s28odHo8pzgbZxAzkkDuOPsWIE5FIW/GLWzuzcSS1o55839DJ4u', 'ROLE_USER', 1);

INSERT INTO `users` (`username`,`password`,`role`,`enabled`)
VALUES ('admin', '$2a$10$mOhFQi0323z.gAZzWvAINugiAseueSXW8PcxM70xlhrZZCCh6ks6a', 'ROLE_ADMIN', 1);
```

Cách cấu hình Spring Security và cấu hình xác thực

```
@Configuration
@EnableWebSecurity
public class WebSecurityConfig extends WebSecurityConfigurerAdapter {

    @Autowired
    private DataSource dataSource;

    @Autowired
    public void configAuthentication(AuthenticationManagerBuilder authBuilder) throws Exception {
        authBuilder.jdbcAuthentication()
            .dataSource(dataSource)
            .passwordEncoder(new BCryptPasswordEncoder())
            .usersByUsernameQuery("select username, password, enabled from users where username=?")
            .authoritiesByUsernameQuery("select username, role from users where username=?")
    }
}
```

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http.authorizeRequests( ) ExpressionUrlAuthorizationConfigurer<...>.ExpressionInterceptUrlRegistry

        .antMatchers( ...antPatterns: "/edit/*", "/delete/*").hasRole("ADMIN")

        .anyRequest().authenticated()

        .and() HttpSecurity
        .formLogin().permitAll() FormLoginConfigurer<HttpSecurity>

        .and() HttpSecurity
        .logout().permitAll() LogoutConfigurer<HttpSecurity>

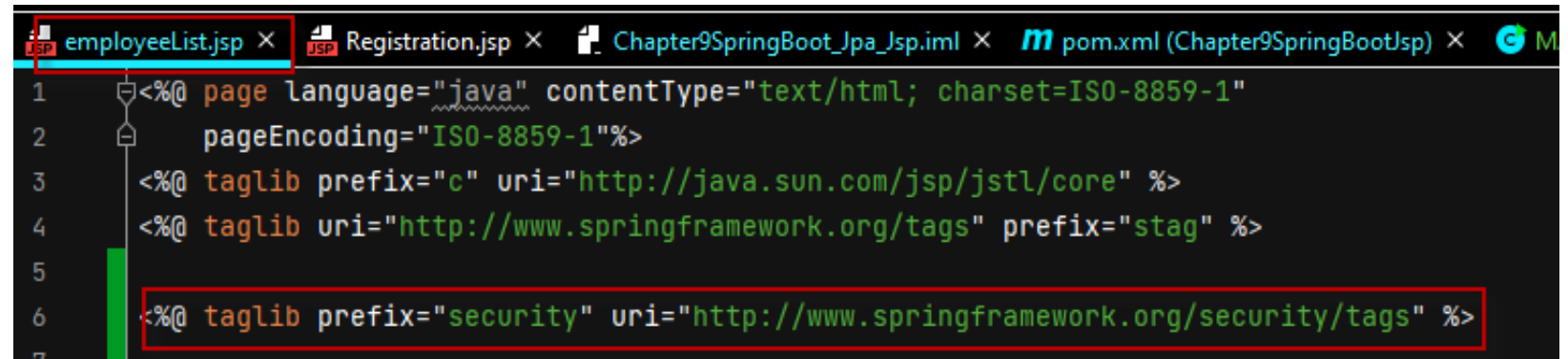
        .and() HttpSecurity
        .exceptionHandling().accessDeniedPage("/403") ExceptionHandlingConfigurer<HttpSecurity>
    ;
}
```

Sử dụng Spring Security Taglibs trong JSP

Thêm dependency

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-taglibs</artifactId>
  <version>5.2.2.RELEASE</version>
</dependency>
```

Khai báo



```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
4 <%@ taglib uri="http://www.springframework.org/tags" prefix="stag" %>
5
6 <%@ taglib prefix="security" uri="http://www.springframework.org/security/tags" %>
7
```

Sử dụng

```
<security:authorize access="hasRole('ADMIN')">
  <td>
    <a href="/edit?id=${emp.id}" >Edit</a>
  </td>
  <td>
    <form action="/delete?id=${emp.id}" method="post">
      <input type="submit" value="Delete" style="..." />
    </form>
  </td>
</security:authorize>
```

Giới thiệu OAuth2

OAuth 2 là một phiên bản tiếp theo của giao thức OAuth (**Open Authorization**), là một tiêu chuẩn được thiết kế để cho phép một **trang web (ứng dụng)** truy cập vào tài nguyên **được lưu trữ bởi các ứng dụng web** khác thay mặt cho người dùng.

OAuth2 xác định có bốn vai trò chính:

Resource Owner (Chủ sở hữu tài nguyên): Chủ sở hữu tài nguyên là người dùng có khả năng cấp quyền truy cập vào tài khoản của họ. Ví dụ: Bạn muốn chia sẻ các thông tin như email, ngày sinh, giới tính, địa chỉ lên một trang web nào đó có thể đăng nhập bằng Facebook. Những thông tin là tài nguyên cần chia sẻ và bạn chính là Resource Owner.

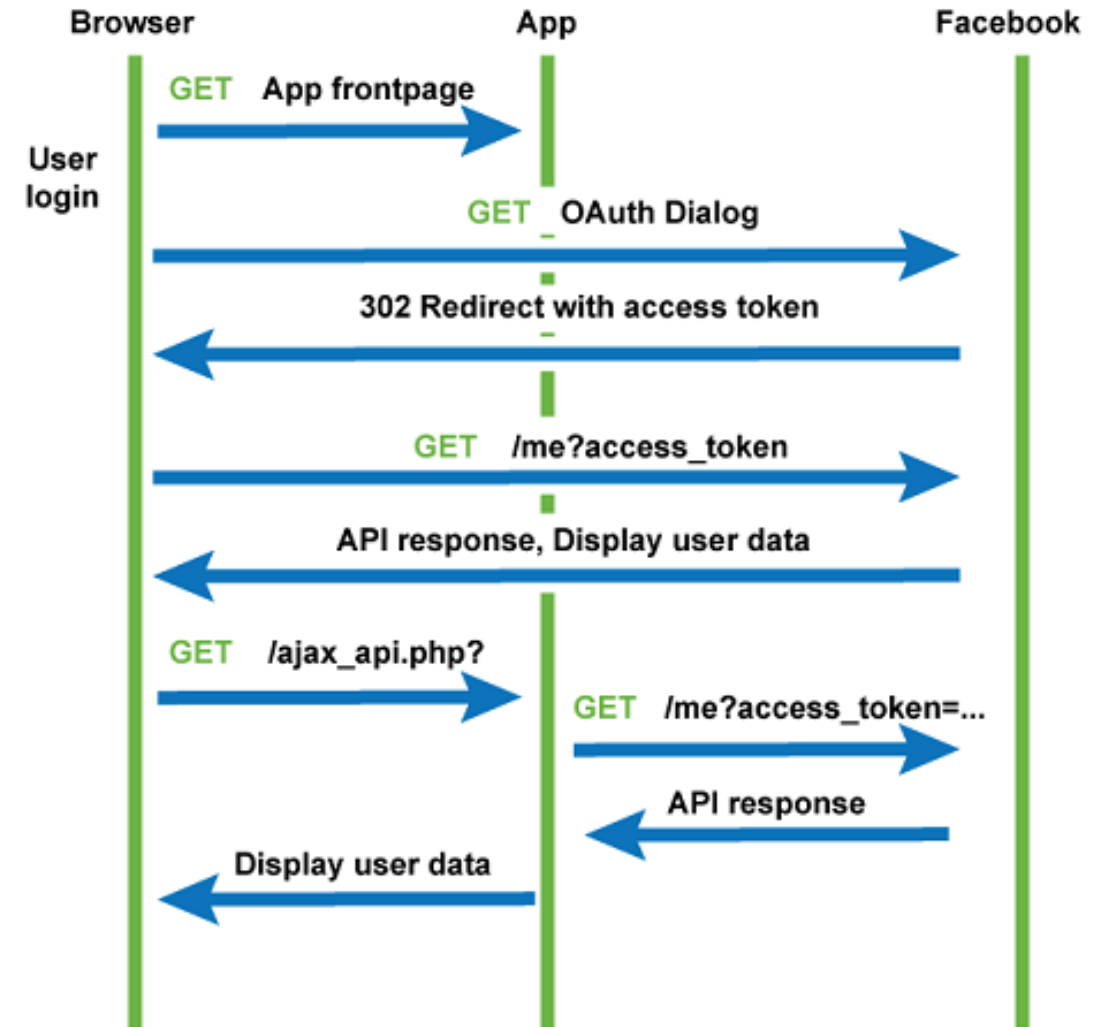
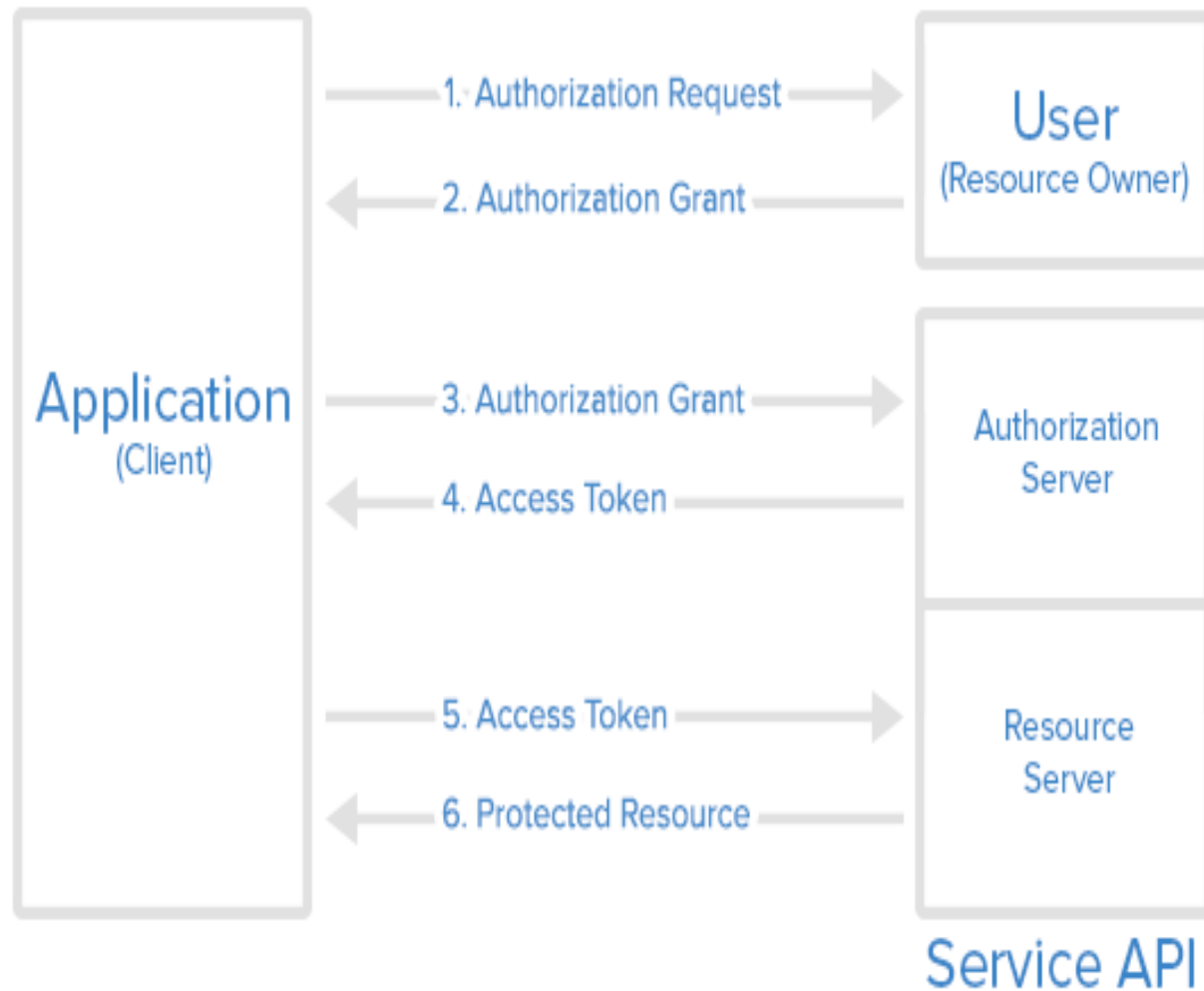
Client (Trang web ứng dụng): Client là ứng dụng muốn truy cập vào tài khoản của người dùng. Trước khi có thể làm như vậy, client phải được người dùng ủy quyền và ủy quyền phải được xác thực bởi API.

Resource Server: Tài nguyên được bảo vệ.

Authorization Server (FB, Google,...): Authorization Server xác minh danh tính của người dùng sau đó cấp mã access token thông báo truy cập cho ứng dụng. Đôi khi Authorization Server cũng chính là resource server.

Luồng xác thực trong OAuth2

Abstract Protocol Flow



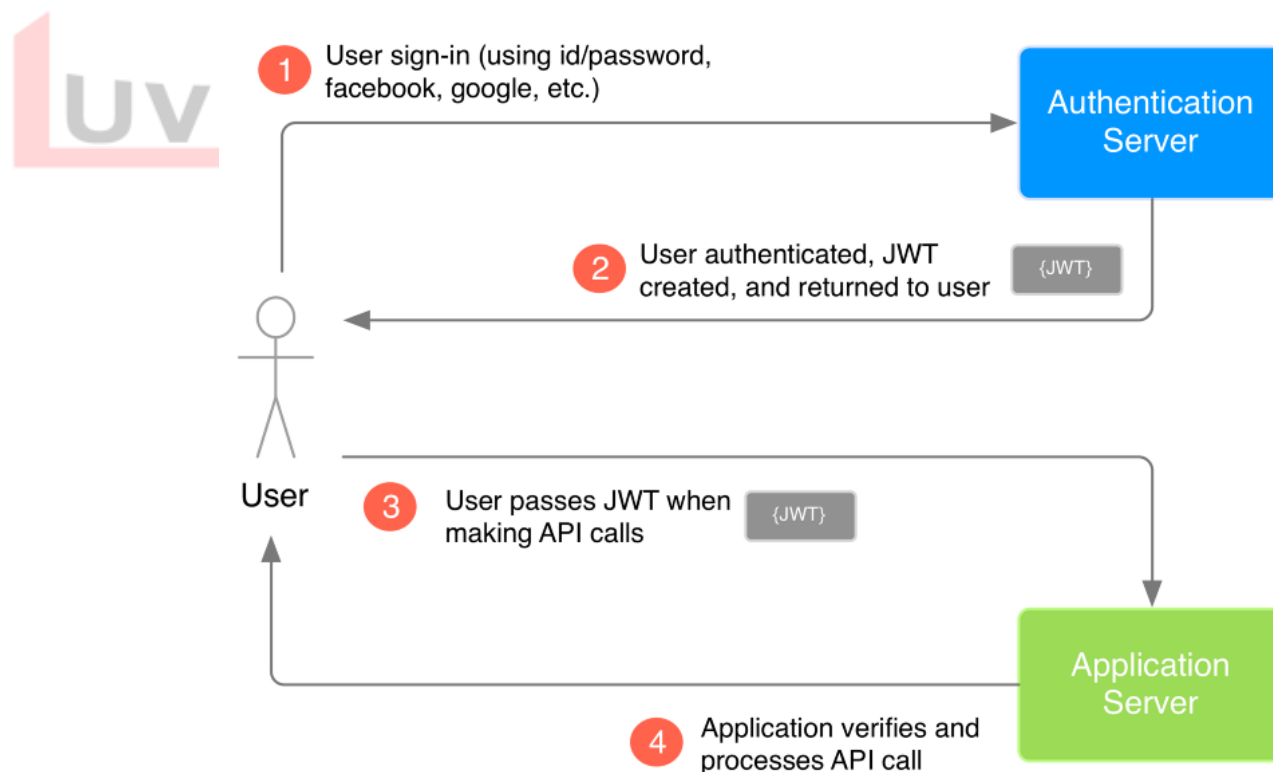
JSON Web Token

Giới thiệu:

Json Web Token (JWT) là một tiêu chuẩn mở (RFC 7519) nhằm xác minh thông tin an toàn giữa các bên Client-Server dưới dạng JSON object.

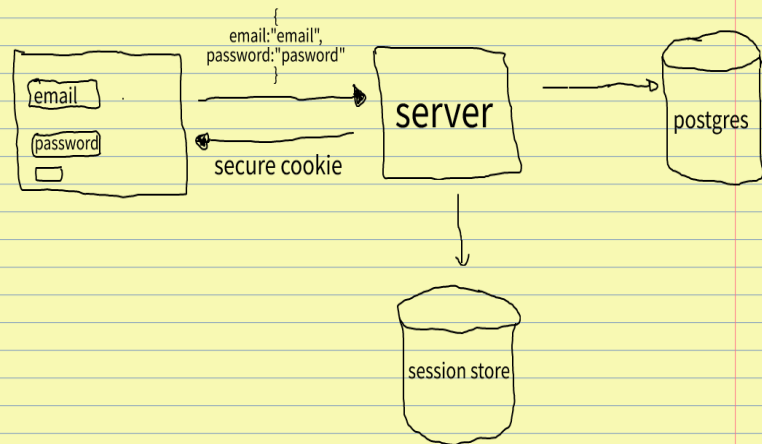
Thông tin này có thể được xác minh và tin cậy vì nó được ký điện tử - digitally signed. JWT có thể được ký bằng cách sử dụng một secret (với thuật toán HMAC) hoặc cặp public/private key dùng chuẩn RSA hoặc ECDSA.

Luồng xác thực sử dụng JWT:

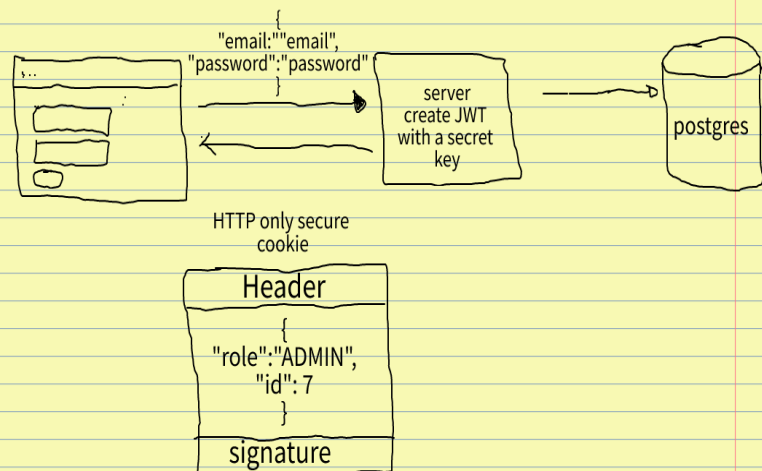


So sánh Session và JWT

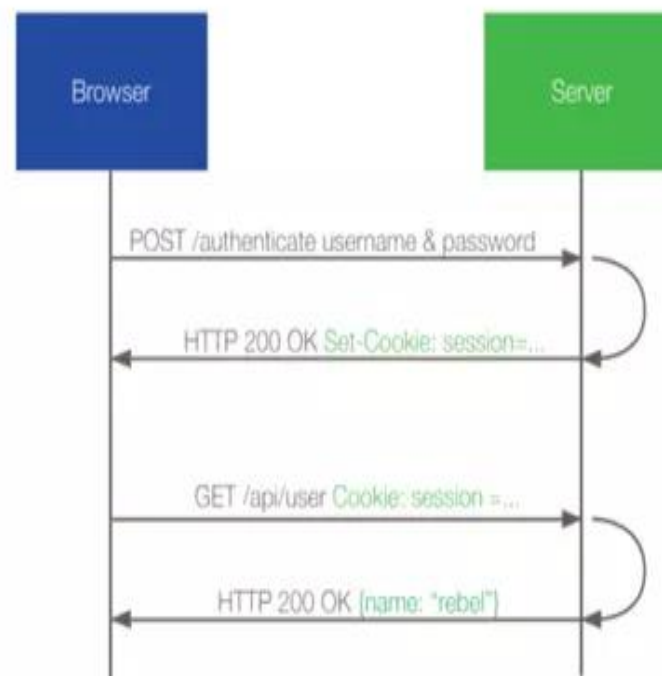
Session



JWT



Traditional Cookie-based Authentication



Modern Token-based Authentication

